AI-Powered Fake News & Deepfake Detection System

1. Introduction

The rapid advancement of artificial intelligence and digital media technologies has significantly increased the spread of misinformation and manipulated content. Fake news articles and deepfake videos pose serious threats to public trust, political systems, and social stability.

This project aims to develop a **web-based AI-powered system** that can automatically detect:

- Fake news from textual data using NLP
- Deepfake videos using computer vision techniques

The system is designed to provide:

- **Real-time predictions**
- **Confidence scores**
- **Basic explanations of results**

By integrating machine learning models with a web interface, this project demonstrates a practical solution to combat misinformation.

2. Objectives

Primary Objectives

- Develop a model to classify news as **Real or Fake**
- Build a system to detect **Deepfake videos**
- Provide **confidence scores** for predictions
- Create a **user-friendly web interface**
- Ensure **fast response time (<3 seconds)**

Secondary Objectives

- Improve model interpretability (why prediction was made)
- Enhance UI/UX design
- Optimize system performance and latency

3. Scope of the Project

Included (MVP Scope)

- Fake news detection using **BERT-based NLP model**
- Deepfake detection using **pretrained CNN/XceptionNet**
- Backend API using **FastAPI**
- Frontend UI (React or HTML)
- Input:
    1. Text input box
    2. Video upload
- Output:
    1. Prediction label
    2. Confidence score

Excluded (Future Scope)

- Real-time video streaming detection
- Browser extensions
- Blockchain verification systems
- Multi-language support

4. System Architecture

High-Level Architecture

User → Frontend → FastAPI Backend → AI Models → Response

5. Detailed Workflow

This is the **core explanation your evaluator will look for**:

Step-by-Step Workflow

Step 1: User Interaction

- User opens the web application
- Chooses one of the options:

1. Enter text (news/article)
2. Upload video file

Step 2: Frontend Processing

- The frontend captures input
- Sends request to backend via API:
    1. /predict/text → for news
    2. /predict/video → for video

Example:

```
POST /predict/text
{
  "text": "Breaking news article..."
}
```

Step 3: Backend (FastAPI)

The backend acts as the **central controller**:

For Text:

1. Receives input text
2. Preprocesses:
    1. Lowercasing
    2. Tokenization
3. Passes text to BERT model

For Video:

1. Receives video file
2. Extracts frames using OpenCV
3. Selects key frames (to reduce computation)
4. Sends frames to deepfake model

Step 4: Model Prediction

Fake News Model (BERT)

- Converts text → embeddings
- Classifies into:
    1. Real
    2. Fake
- Outputs probability score

Deepfake Model

- Analyzes facial patterns in frames
- Detects inconsistencies
- Outputs frame-level predictions

Step 5: Aggregation Logic

For video:

Final Prediction = Average (frame predictions)

Example:

- Frame predictions: [0.8, 0.9, 0.7]
- Final confidence = 0.8

Step 6: Response Generation

Backend returns:

```
{
  "label": "Fake",
  "confidence": 0.91
}
```

Step 7: Frontend Display

- Displays result:
    1. **Label (Fake/Real)**
    2. **Confidence (%)**

- Optional:
    1. Explanation text

6. Functional Requirements

🔹 **Fake News Detection**

- Input: Text
- Processing: NLP model
- Output:
    1. Label: Real / Fake
    2. Confidence score

🔹 **Deepfake Detection**

- Input: Video (.mp4)
- Processing:
    1. Frame extraction
    2. Model prediction
- Output:
    1. Label: Real / Deepfake
    2. Confidence

🔹 **Dashboard**

- Display results clearly
- Show confidence percentage
- Provide simple explanation

7. Non-Functional Requirements

Requirement	Target
Response Time	< 3 sec
Accuracy	80–85%
Scalability	API-based
Usability	Simple UI

8. Technology Stack

Backend

- FastAPI (Python)

Machine Learning

- HuggingFace Transformers (BERT)
- OpenCV
- Pretrained Deepfake Model

Frontend

- React / HTML / JavaScript

Database (Optional)

- MongoDB

9. Project Structure

project/

|

├── backend/

| ├── main.py

| ├── models/

| | ├── fake_news_model.py

| | ├── deepfake_model.py

| ├── utils/

| | ├── preprocess.py

|

├── frontend/

| ├── src/

| | ├── App.js

|

├── data/

├── notebooks/

├── requirements.txt

├── README.md

10. Methodology

Phase 1: Fake News Detection

- Load dataset (FakeNewsNet / CSV)
- Train BERT classifier
- Evaluate using:
    1. Accuracy
    2. F1-score

Outcome:

- Model accuracy: ~80–85%
- Saved model for inference

Phase 2: Deepfake Detection

- Extract video frames

- Use pretrained model
- Predict per frame

Outcome:

- Functional deepfake detection
- Accuracy: ~70–80%

Phase 3: Backend Development

- Build APIs
- Handle requests
- Integrate models

Outcome:

- Fully working backend

Phase 4: Frontend

- Build UI
- Connect API

Outcome:

- Working interface

Phase 5: Integration

- End-to-end testing

Outcome:

- Complete working system

11. Testing Strategy

Unit Testing

- Model outputs
- API endpoints

Integration Testing

- Full pipeline testing

Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1-score

12. Deployment Plan

Local Deployment

uvicorn main:app --reload

Optional

- Docker
- Cloud platforms

13. Risks & Mitigation

Risk	Solution
No GPU	Use pretrained models
Slow video processing	Reduce frames
Dataset issues	Use clean datasets
Time shortage	Focus on NLP

14. Timeline

Day	Task
1–2	NLP model
3–4	Deepfake
5–6	Backend
7–8	Frontend
9	Integration
10	Testing

15. Expected Outcomes

Guaranteed:

- Working fake news detector
- API-based system
- Functional UI

Partial:

- Deepfake detection (basic)
- UI polish

Not Guaranteed:

- Production-level accuracy
- Real-time video

16. Conclusion

This project successfully demonstrates how AI can be used to detect misinformation in both text and video formats. By focusing on an MVP approach, the system achieves practical functionality within a limited time frame.

It serves as a strong foundation for future improvements such as real-time detection, multi-language support, and enhanced explainability.